DTU Compute
Department of Applied Mathematics and Computer Science
Advanced Programming (02324), Spring 2026
Carlos E. Budde and Ekkart Kindler

Assignment 3: Generic Sorted Lists & Java Collections

Assignment 3 revises and extends all you have learnt for Stacks, by challenging you to develop implementations for an interface of another main abstract data type—Lists:

- *in **Assignment 3a** (Friday, February 13) you will use arrays to implement the essential functionality of a generic `List<E>`, as defined in the interface provided to you;*
- *in **Assignment 3b** (Tuesday, February 17) you will implement an algorithm to sort the elements inside the list, plus the data type `SortedList<E>` which is not quite the same;*
- *in **Assignment 3c** (Friday, February 20) you will extend the GUI that operates with these implementations, using Java Collections to add useful functionality.*

These assignments are incremental, meaning that they build on each other. You start by downloading the IntelliJ/Maven project `assignment3.v1.1.0.zip` from DTU Learn. Assignment 3a begins from those base files; once you have finished and presented your solution, Assignment 3b continues from that solution; ditto Assignment 3c.

Assignment 3c: Java (Hash)Map, and GUI primer

In Assignment 3c, you must provide extensions to your List implementations that lets users specify the weight of the `Person` inserted. For this, you should extend the GUI with a numeric input field that captures `weight` values. You will also extend the GUI to insert a `Person`—its `name` and `weight`—at a user-specified `index`. Finally, you will compute and show the average `weight` of all `Persons` in the list, as well as the `name` that appears repeated the highest number of times, using a Java `Map<K,V>` as shown in class. More in detail:

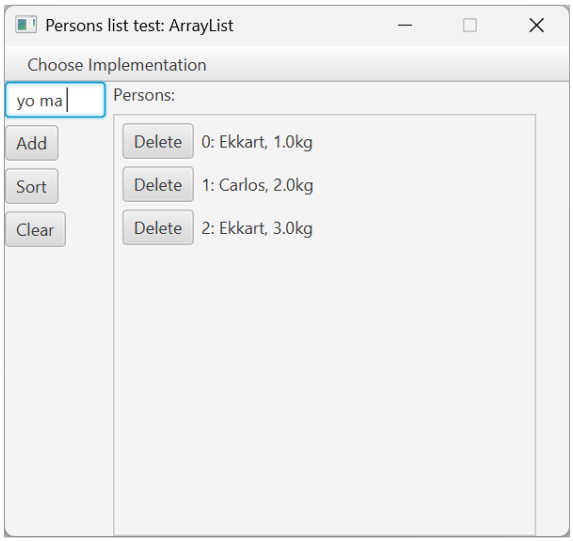
1. Before you start, a group member must have presented and approved Assignment 3b.
   - Your work for Assignment 3c starts from your solution presented for Assignment 3b.
2. Add a `TextField` to the GUI, where users specify the weight of the `Person` being added.
   - Leverage the code you are given: read `uses/PersonsGUI.java` from Assignment 3, and the `IntegerStackGUI` constructor from Assignment 2, to see how a `TextField` is created in our JavaFX GUI to let users fill in the `name` of the `Person` to add.
   - Add an equivalent `TextField` for `weight`, which must be a *numeric value*. For this:
     - Either you implement the solution from Assignment 2, where the user is only allowed to input valid numeric values;
     - Or else, allow the user to input any text, and validate internally after "Add" is pressed. Moreover, you can either force users to provide a `weight`, e.g. throw an Exception when they don't; or insert a default value when a `weight` is not given.

3. Add another numeric `TextField` to the GUI, together with a new `Button` called "Add at index:", to insert a Person at a given index.
   - Read `PersonsGUI.java` to see how a `Button` is created in the GUI.
   - The expected behaviour is the same as the `add(int pos, E e)` method of a `List`.

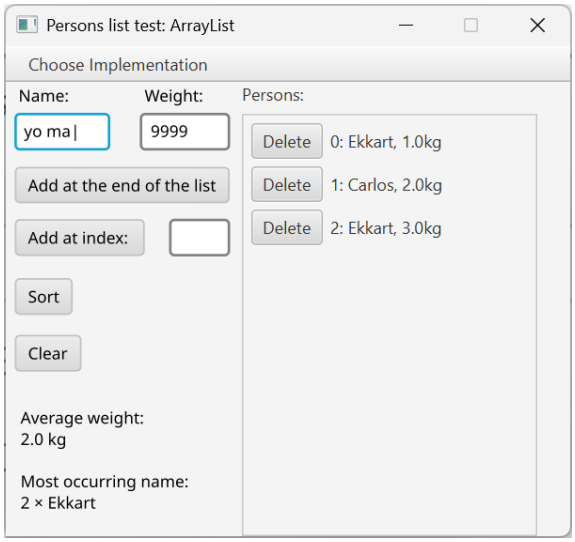
4. Add two `Labels` to the GUI, that display the average weight of all Persons currently in the list, as well as the most frequently occurring `name`. These are updated as below:
   - Read `PersonsGUI.java` to see how a `Label(String s)` is created in the GUI.
   - Create Labels that you will use to display the average of weights—i.e. the mean of all weights in your list—and the most frequently occurring name—see the example of `Map<K,V>` in Lecture 06 slides for an idea on how to count frequency of occurrence.
   - Each time the list is modified, update() these values in the GUI.

5. The `// TODO ...` comments in `PersonsGUI.java` will help you identify the locations where your code should be added for the tasks listed above. An illustrative example of how the final GUI could look is shown as the "New" image below (right):

Old and busted



New hotness



6. The solution can be considered complete when all functionality above has been implemented and tested.
   - Note that the GUI should be tested by hand, including exceptional cases such as invalid input like negative weight, invalid index, and trying to sort a `SortedList`.

7. The solution should be shown to the teachers or teaching assistants during the tutorial sessions on at latest on Tuesday, February 24.
   - If you can, you're strongly advised to present on Friday, February 20, since the group submission for Part 1 of the course depends on this solution.

8. If you have time left ***after presenting Assignment 3c***, you can extend the GUI further:
   - Add a text area that shows any exceptions thrown, e.g. if the user attempts to insert an element at an invalid index. The expected functionality is the one that you had for Assignment 2b, where the exceptions thrown were caught and shown in the text area (instead of the IDE console).
   - Add buttons to return the `indexOf(Person p)` and whether the list `contains(Person p)` for a user-input "`name,weight`". Show the results of these queries via a `Label` in the GUI, and define how to deal with invalid input.

References and further reading

[02324 f25 L04, L05, L06] Carlos E. Budde and Ekkart Kindler: Lecture Notes for the course Advanced programming. Spring 2025 (DTU Learn).

[Eck 2022] David J. Eck: Introduction to Programming Using Java. Version 9.0, JavaFX Edition, May, 2022. Sections 7.5, 10.2–10.4.